

Algorithmic

1. What is an Algorithm?

1. Basics
2. History
3. Parts

2. Branchings

1. Predicates
2. Syntax
3. Dependent conditions
4. Comparison
5. Logical operators
6. Nesting
7. Keyword In

3. Loops

1. For loops
2. zip & enumerate
3. any & all
4. List comprehensions
5. While loops
6. Nested Loops

4. Visualizing source code

5. Definitions

What is an Algorithm?

- An algorithm is a complex of related commands, that serve a specific purpose or fulfill a specific task
- Algorithms are instructions for the systematic solution of a problem
- Tries to guarantee good or best possible results
- More advanced algorithms can use conditionals to divert the code execution through various routes
- Sometimes optimal solutions would take too long, in these case sub optimal solutions generated by heuristic algorithms can be an alternative

- Step-by-step procedures for solving mathematical problems have been attested since antiquity:
 - ▶ Babylonian mathematics around 2500 BC
 - ▶ Egyptian mathematics around 1550 BC
 - ▶ Indian mathematics around 800 BC
 - ▶ Greek mathematics around 240 BC
 - ▶ Arabic mathematics 9th century
- The term itself stems from Muhammad bin Musa al-Khwarizmi [2] who used the term for the first time around 825 for his book *kitab al-jam' wa'l-tafriq al-ḥisāb al-hindī* ("Addition and subtraction in Indian arithmetic")



- The first time an algorithm was created in combination with computers was in 1843 when Ada Lovelace developed the very first computer program
- From that day on programming and algorithms went hand in hand

Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 et seq.)

Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data												Working Variables.												Result Variables.			
						V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8	V_9	V_{10}	V_{11}	V_{12}	V_{13}	V_{14}	V_{15}	V_{16}	V_{17}	V_{18}	V_{19}	V_{20}	V_{21}	V_{22}	V_{23}	V_{24}	V_{25}	V_{26}	V_{27}	
						1	2	n																									
1	X	$V_2 \times V_3$	V_6, V_7, V_8	$V_6 = V_2 \times V_3$	$V_6 = 2n$	...	2	n	2n	2n	2n																						
2	-	$V_4 - V_5$	V_9	$V_9 = V_4 - V_5$	$V_9 = 2n - 1$	1	...	...	2n - 1																								
3	+	$V_6 + V_7$	V_{10}	$V_{10} = V_6 + V_7$	$V_{10} = 2n + 1$	1	...	...	2n + 1																								
4	+	$V_6 + V_7$	V_{11}	$V_{11} = V_6 + V_7$	$V_{11} = 2n + 1$				0	0																							
5	+	$V_{10} + V_{11}$	V_{12}	$V_{12} = V_{10} + V_{11}$	$V_{12} = 2n + 1$	2	...	...																									
6	-	$V_{12} - V_{11}$	V_{13}	$V_{13} = V_{12} - V_{11}$	$V_{13} = 1$																												
7	-	$V_{12} - V_{11}$	V_{14}	$V_{14} = V_{12} - V_{11}$	$V_{14} = 1$	1	...	n																									
8	+	$V_2 + V_7$	V_{15}	$V_{15} = V_2 + V_7$	$V_{15} = 2 + 0 = 2$	2	...																										
9	+	$V_6 + V_7$	V_{16}	$V_{16} = V_6 + V_7$	$V_{16} = 2n$																												
10	X	$V_{15} \times V_{16}$	V_{17}	$V_{17} = V_{15} \times V_{16}$	$V_{17} = 2n$																												
11	+	$V_{17} + V_{16}$	V_{18}	$V_{18} = V_{17} + V_{16}$	$V_{18} = 2n + 1$																												
12	-	$V_{18} - V_{17}$	V_{19}	$V_{19} = V_{18} - V_{17}$	$V_{19} = 1$	1	...																										
13	+	$V_6 - V_7$	V_{20}	$V_{20} = V_6 - V_7$	$V_{20} = 2n - 1$	1	...																										
14	+	$V_6 + V_7$	V_{21}	$V_{21} = V_6 + V_7$	$V_{21} = 2n + 1$	1	...																										
15	+	$V_6 + V_7$	V_{22}	$V_{22} = V_6 + V_7$	$V_{22} = 2n + 1$																												
16	X	$V_{22} \times V_{21}$	V_{23}	$V_{23} = V_{22} \times V_{21}$	$V_{23} = 2n$	1	...																										
17	-	$V_{23} - V_{22}$	V_{24}	$V_{24} = V_{23} - V_{22}$	$V_{24} = 1$																												
18	+	$V_6 + V_7$	V_{25}	$V_{25} = V_6 + V_7$	$V_{25} = 2n + 1$	1	...																										
19	+	$V_{25} + V_{24}$	V_{26}	$V_{26} = V_{25} + V_{24}$	$V_{26} = 2n + 1$																												
20	X	$V_{26} \times V_{25}$	V_{27}	$V_{27} = V_{26} \times V_{25}$	$V_{27} = 2n$																												
21	+	$V_{27} + V_{26}$	V_{28}	$V_{28} = V_{27} + V_{26}$	$V_{28} = 2n + 1$																												
22	+	$V_{28} + V_{27}$	V_{29}	$V_{29} = V_{28} + V_{27}$	$V_{29} = 2n + 1$																												
23	-	$V_{29} - V_{28}$	V_{30}	$V_{30} = V_{29} - V_{28}$	$V_{30} = 1$	1	...																										
Here follows a repetition of Operations thirteen to twenty-three.																																	
24	+	$V_{30} + V_{29}$	V_{31}	$V_{31} = V_{30} + V_{29}$	$V_{31} = 2$																												
25	+	$V_{31} + V_{30}$	V_{32}	$V_{32} = V_{31} + V_{30}$	$V_{32} = 4$	1	...	n + 1																									

- As we learned an algorithm contains a series of commands that aim to fulfill a specific task
- These commands need to be structured to be able to react to the value of our variables as well as the results of our calculations
- Furthermore they need to be able to repeat certain processes multiple times
- So that our algorithms can handle these tasks several programming constructs were developed that help us with them
- These constructs are:
 - Branchings or if-statements
 - Loops

Branchings

- Opportunity to execute only specific blocks of code, while excluding other blocks of code
- Mostly done by using IF clauses, which can be supplemented with ELSE but don't have to
- They use predicates / conditions to determine which part of the source code should be executed
- Allow the program to react to different results, files and/or inputs to make the script in itself more robust
- Play an essential role in avoiding errors by using so called security checks that are included in scripts to make sure all calculations are possible at each time

- There are many different ways of using Branchings but some of the most common ones are the following:
 - ▶ Check whether an input is a number or not
 - ▶ Check whether a certain file exists
 - ▶ Checking if a calculation result fulfills certain criteria
 - ▶ Checking if a value is an element of a data structure
 - ▶ Making sure that a datastructure contains at least a certain amount of elements
 - ▶ Searching for substrings in strings / files
 - ▶ Security Checks to make sure your variables can be used for the planned calculations

if (Predicate):

 instructions1

 instructions1

else:

 instructions2

➔ We don't need to put the predicate in brackets, here it was just done for explanatory reasons

- A statement that can be either true or false
- So we could say the answer is always yes or no
- In all cases the whole predicate is of the type bool
- You can link predicates with logical operators like OR and AND
- We can negate predicates by using NOT
- There are different types of predicates we can create:
 - simple mathematic comparisons using $>$ or $<$
 - Testing for equalness using $==$
 - Searching for elements in data structures using in

- Let's say we have a variable called number1 and depending on the value of this variable we want to print out different things
- To do something like this we need to create an if statement
- In the first step we need to generate the predicate which tells the interpreter later on which part of the source code should be executed
- For our example we want to run a specific print out as soon as the value of the variable number1 is bigger than 20
- That means the predicate is $\text{number1} > 20$
- Now we combine this with if to create our first line of the branching:

```
if number1 > 20:
```

- As you could see I ended the line of my if statement with a colon (“:”), whenever we create an if statement (also true for elif and else as we will see later), a loop or when define something like a function we need to close the corresponding line with such a colon
 - ▶ This is the only form of punctuation Python needs
- As soon as we created an if statement, or any other line that ends with a “:”, we need to write at least one line of additional code directly afterwards
- When we forget this, we will get errors and our script is not able to be executed

- Since we need at least one line of code after the if statement we can now write down the following code:

```
if number > 20:  
    print("A big number")
```

- As you can see the line with the print statement starts further on the right than the if statement
- This is called indentation and tells Python that this line is inside our if statement and thus has to be executed as soon as our predicate ($\text{number} > 20$) becomes True

- We always have to indent lines that are inside our if statement and only lines that are indented like this and start at the same level are than seen as inside our if statement
- Please use the same level of indentation (four spaces, two spaces, a tab stop, etc.) through out your whole script to make it more readable
- Per style guide you should use four spaces as indentation
- As soon as Python encounters a line that is NOT indented any more it knows that we are now outside of our if statement
- Our branching is now complete but we could supplement it with an else, to include source code that is executed as soon as our predicate is False

- else statements are NOT necessary but can be useful
- To supplement our if statement with an else, we just need to add else and at least one additional line of source code to our branching:

```
if number > 20:  
    print("A big Number")  
  
else:  
    print("A small Number")
```

- As you can see the else starts at the same position as the if statement and like this Python knows that this else supplements the if before
- Please keep in mind that else can NOT be used without an if it supplements

```
number = 30
```

➔ number contains 30

```
if number > 20:
```

➔ Computer asks 30 > 20?

```
    print("A big number")
```

```
else:
```

```
    print("A small number")
```

```
print("Finished")
```

```
number = 10
```

➔ number contains 10

```
if number > 20:
```

➔ ~~Convert~~ Masks 10 > 20?

```
    print("A big number")
```

```
else:
```

```
    print("A small number")
```

```
print("Finished")
```

- Write a program that contains at least three different if/else statements
- Try to compare numbers as well as strings

- Python allows you to make multiple different checks in a row by using the if, elif, else structure
- Here the predicates are tested one after the other and as soon as one of them becomes True, only the source code that is inside the corresponding if or elif is executed
- That means if Predicate1 is False and Predicate2 is True we only execute instructions2
- On the other hand if Predicate1 is True we only execute instructions1 even though Predicate2 and Predicate3 could also be True

```
if (Predicate1):  
    instructions1  
  
elif (Predicate2):  
    instructions2  
  
elif (Predicate3):  
    instructions3  
  
else:  
    instructions
```

- The aim for using this construct (if, elif, else) is to run consecutive checks that are dependent from each other
- For example if we would use:

```
if a+b > 100: ...
```

```
if a+b > 10: ...
```

```
if a+b > 5: ...
```

- With $a = 60$ and $b = 50$ we would get true for all three statements, but we only want to run instructions for one of the three statements

```
number1 = 50
```

➔ number1 contains 50

```
number2 = 60
```

➔ number2 contains 60

```
if number1 + number2 > 100:
```

➔ Computer asks 50+60 > 100?

```
    print("A very big number")
```

```
elif number1 + number2 > 50:
```

```
    print("A big number")
```

```
elif number1 + number2 > 10:
```

```
    print("A mediocre number")
```

```
else:
```

```
    print("A small number")
```

```
number1 = 50
```

➔ number1 contains 50

```
number2 = 30
```

➔ number2 contains 30

```
if number1 + number2 > 100:
```

➔ Computer asks 50+30 > 100?

```
    print("A very big number")
```

```
elif number1 + number2 > 50:
```

➔ Computer asks 50+30 > 50?

```
    print("A big number")
```

```
elif number1 + number2 > 10:
```

```
    print("A mediocre number")
```

```
else:
```

```
    print("A small number")
```

```
number1 = 5
```

➔ number1 contains 5

```
number2 = 30
```

➔ number2 contains 30

```
if number1 + number2 > 100:
```

➔ Computer asks 5+30 > 100?

```
    print("A very big number")
```

```
elif number1 + number2 > 50:
```

➔ Computer asks 5+30 > 50?

```
    print("A big number")
```

```
elif number1 + number2 > 10:
```

➔ Computer asks 5+30 > 10?

```
    print("A mediocre number")
```

```
else:
```

```
    print("A small number")
```

```
number1 = 5
```

➔ number1 contains 5

```
number2 = 3
```

➔ number2 contains 3

```
if number1 + number2 > 100:
```

➔ ~~Computer~~asks 5+3 > 100?

```
    print("A very big number")
```

```
elif number1 + number2 > 50:
```

➔ ~~Computer~~asks 5+3 > 50?

```
    print("A big number")
```

```
elif number1 + number2 > 10:
```

➔ ~~Computer~~asks 5+3 > 10?

```
    print("A mediocre number")
```

```
else:
```

```
    print("A small number")
```

- Write a program that defines a string variable name
- If the string is your name print "That is a nice name", if it's "John Cleese" or "Michael Palin", print something to tell how you feel about them ;), otherwise print "You have a nice name."
- Do all of this using an if, elif, else construct

- There are 6 possible mathematical expressions that can be used to compare things
 - $<$ → less than
 - $<=$ → less than or equal
 - $>$ → greater than
 - $>=$ → greater than or equal
 - $==$ → equal
 - $!=$ → not equal
- We can use all operators on the right side with numbers and strings
- Comparisons of Strings are done in form of case-sensitive absolute matches
- We can not compare strings to numbers

- We saw that we can use variables in our if statements to check for their corresponding value
- Of course we can also compare variables with each other using the comparison operators we just saw
- Imagine we have two variables called num1 and num2
- Now we want to check whether they have the same value or not, in this case we can create a simple if statement as follows:

```
if num1 == num2:  
    print("These numbers are identical")
```

- Write a program that defines the variables var1, var2, var3, var4, var5 and var6 with the type of your choice
- Create six if statements to test the following aspects:
 - ▶ Is var1 bigger than var4
 - ▶ Is var2 smaller than var5
 - ▶ Is var3 equal to var6
 - ▶ Is var4 bigger than or equal to var 2
 - ▶ Is var 5 smaller than or equal to var3
 - ▶ Is var6 not equal to var1

- Until now we only used one single predicate for our if statements
 - We are however able to combine (or negate) them using different logical operators
 - There are two logical operators available to us for combining:
 - and
 - or
- Their general usage is as follows:
if (Predicate1 and Predicate2):
 instructions
 - if (Predicate1 or Predicate2):
 instructions
 - You're not limited to one operator per if:
if (Predicate1 and Predicate2 and Predicate3):
 instructions

- Predicate1 and Predicate2
 - ➔ only true if both Predicates are True
- Predicate1 or Predicate2
 - ➔ true if one Predicate is True

```
num1 = 10
```

➔ num1 contains 10

```
num2 = 10
```

➔ num2 contains 10

```
num3 = 8
```

➔ num3 contains 8

```
num4 = 8
```

➔ num4 contains 8

```
if num1 == num2 and num3 == num4:
```

➔ ~~10==10 AND 8=8?~~

```
    print("Two pairs of identical numbers")
```

```
else:
```

```
    print("Too bad!")
```

```
num1 = 10
```

➔ num1 contains 10

```
num2 = 10
```

➔ num2 contains 10

```
num3 = 8
```

➔ num3 contains 8

```
num4 = 9
```

➔ num4 contains 9

```
if num1 == num2 and num3 == num4:
```

➔ ~~10==10 AND 8=9?~~

```
    print("Two pairs of identical numbers")
```

```
else:
```

```
    print("Too bad!")
```

```
num1 = 10
```

➔ num1 contains 10

```
num2 = 10
```

➔ num2 contains 10

```
num3 = 8
```

➔ num3 contains 8

```
num4 = 9
```

➔ num4 contains 9

```
if num1 == num2 or num3 == num4:
```

➔ 10==10 OR 8==9?
Answer: Yes

```
    print("Two pairs of identical numbers")
```

```
else:
```

```
    print("Too bad!")
```

- Write a program that defines two numerical variables called number1 and number2
- Create if statements to test the following:
 - ▶ Are both numbers greater than 0
 - ▶ Are both numbers smaller than 0
 - ▶ Is at least one number greater than 0
 - ▶ Is at least one number smaller than 0
 - ▶ Is one number greater than 0 and the other one smaller than 0
 - ▶ Are both numbers in the range of -10 to 10

- Another logical operator we can use is the operator not
- This operator allows us to negate a predicate in our if statement
- Mostly used in combination with the keyword in (we will talk about this a bit later) but could be used for normal predicates as well
- A simple example would be as follows:

```
if not num1 == num2:
```

```
    print("These numbers are different")
```

- In the example below the Predicate becomes True as soon as num1 and num2 have different values
- As you can see we negated the predicate and thus switched the logic

- Working with not can get confusing very fast
- Therefore try to avoid using this operator whenever you can, since the reversed logic it produces is something that can lead to many troubles later down the line

- Write a program that defines two numerical variables called number1 and number2
- Create if statements containing the logical operator not to test the following:
 - ▶ Is number1 smaller than 0
 - ▶ Is number2 bigger than 0
 - ▶ Is number1 outside of the range of -10 to 10
 - ▶ Is number2 smaller than -100
 - ▶ Is number1 bigger than 250
 - ▶ Are both numbers bigger than 0

- We saw when talking about datastructures that we can put them into each other by using a process called nesting
- The same process can be used with our programming constructs (if statements and loops) as well
- By nesting if statements into each other we generate a logical order of checks and can model any dependencies between our Predicates
- The logic of nesting if statements differs a bit from combining predicates using logical operators as we will see on the next slides

- Instead of using **and** to combine if-clauses, we could also nest them:

if (Predicate1) and (Predicate2):

 Instructions

➔ This version should be used if all instructions you want to execute are dependent from both Predicates being True

if (Predicate1):

 Instructions1

 if (Predicate2):

 Instructions2

➔ This version should be used if only some instructions you want to execute are dependent from both Predicates being True and you have some instructions you want to execute independently from Predicate2 being True or False while they still depend on Predicate1 being True

- When creating nested if statements we need to create our Predicates first
- Now we need to analyze the logic of our Predicate and the instructions we want to run for each predicate
- Let's say we have three Predicates called Pred1, Pred2 and Pred3 and we have three blocks of instructions Ins1, Ins2 and Ins3
- When the instruction block Ins1 is only dependent on Pred1 to be true while Ins2 and Ins3 need Pred2 and Pred3 to be true as well then we can create a nested if statement

- Imagine our Instructions have the dependencies shown in the table below, that means for each new block another predicate needs to be true
- As soon as we have something like this we can use nested ifs:

if Pred1 is true:

 Ins1

 if Pred2 is true:

 Ins2

 if Pred3 is true:

 Ins3

Instruction	Dependencies
Ins1	Pred1 is true
Ins2	Pred1 and Pred2 are true
Ins3	Pred1, Pred2 and Pred3 are true

- As you could saw on the previous slide with each if the indentation of the following instruction block increased and we created a “Stairs-like” structure of our source code
- Like this Python knows that one if is inside another if and therefore only when the first if was True the second if is checked
- So please be careful with your indentation to make sure that the logic you want to achieve is represented by your source code

```
num1 = 10
```

➔ num1 contains 10

```
num2 = 10
```

➔ num2 contains 10

```
num3 = 8
```

➔ num3 contains 8

```
num4 = 8
```

➔ num4 contains 8

```
if num1 == num2:
```

➔ 10==10? Answer: Yes

```
    print("One pair of identical numbers")
```

```
    if num3 == num4:
```

➔ 8==8? Answer: Yes

```
        print("Two pairs of identical numbers")
```

```
num1 = 10
```

➔ num1 contains 10

```
num2 = 10
```

➔ num2 contains 10

```
num3 = 8
```

➔ num3 contains 8

```
num4 = 9
```

➔ num4 contains 9

```
if num1 == num2:
```

➔ 10==10? Answer: Yes

```
    print("One pair of identical numbers")
```

```
    if num3 == num4:
```

➔ 8==9? Answer: No

```
        print("Two pairs of identical numbers")
```

- Working with nested ifs can be a bit complicated
- However sometimes there is no other way than using such complicated constructs to model a correct solution
- One example when these nested ifs are necessary is when you work with files
- Here we need to go over several steps to get to the correct data:
 1. Test whether the file exists → First if
 2. Read in the file → Inside first if
 3. Test whether contents are present → Second if
 4. Go over contents → Inside second if
 5. Test for certain keywords → Third if

- Another example comes when working with sequences:
 1. Test for length of sequence → First if
 2. Read in sequence → Inside first if
 3. Test for sequence type → Second if
 4. Statistical analysis → Inside Second if
- As you can see from these two rather simple examples, the usage of nested ifs is something that will appear relatively often in bioinformatics

- Write a program that defines two numerical variables called number1 and number2
- Create nested if statements to test the following:
 - ▶ Are both numbers greater than 0
 - ▶ Are both numbers smaller than 0
 - ▶ Is number1 equal to 0 and number2 not equal to 0
 - ▶ Is number1 bigger than 0 and number2 smaller than 0
 - ▶ Are both numbers in the range of -50 to 50
- Please include a print out after each if statement.

- When talking about datastructures I mentioned that we can create checks to see whether an element is inside our datastructure or not
- We can do this with the help of if statements and the keyword in
- For lists / tuples / sets the syntax is very simple and we can use a statement as shown below (exemplary for a list)

if element in list:

instructions

- Write a program that generates a list with contents of your choice
- Create three if statements to check whether a value of your choice is part of your list or not
- Print out an appropriate answer

- When working with dictionaries we have two options for what we want to search in them
- If we are interested in whether a key exists in our dictionary or not we can use:
 if key in dictionary:
 instructions
- If we want to search for a specific value in our dictionary than we have to use the method `values()`:
 if value in dictionary.values():
 instructions

- Create a dictionary with at least three key/value pairs
- Define two variables (one key and one value)
- Search the dictionary for the specified key
- If the key is already present tell the user that you update the key/value pair
- If the key does not exist tell the user that it's added to the dictionary
- Use an assignment to update/add the key/value pair to your dictionary
- Run the same code at least three times during one call of the program
- Print out the original dictionary at the beginning and the modified dictionary at the end

Loops

- When creating programs you will run into situations where your program needs to do a similar task over and over again
- Of course you could type in the corresponding source code let's say 10 times to fulfill this task but this would be troublesome, error-prone and your program would be unnecessarily long
- Therefore programmers developed tools like loops or functions to reduce the amount of typing as well as to increase the readability of their programs

- Loops are programming constructs that allow us to repeat source code a (in)finite number of times
- We do this by using a control or loop variable that is assigned a different value each time our source code is repeated
- These repetitions are called iterations and the term iterable for data structures stems from this as well
- Generally speaking there are two type of loops in Python:
 - For – Loop
 - While – Loop

- This type of loops runs a definite amount of iterations before the program goes on with the next task
- We specify the number of iterations when creating our loop and therefore it's nearly impossible to create an endless loop
- For loops in Python are so called Collection-Based or Iterator-Based Loops and iterate over a collection of values
- The general construction of a for loop in Python looks like this

for var in iterable:

instructions

- Let's go over the different parts of a for loop constructor
- First the keyword “for”, this keyword tells the interpreter that we want to create a for loop, and thus needs to be used whenever we want to create such a loop
- Second the loop variable var, whose name we define as we see fit
 - ▶ There are some conventions regarding the naming we will talk about in a bit
 - ▶ Whenever Python encounters our constructor for the first time the loop variable var gets declared and assigned a value in the form of `var=iterable[0]`
 - ▶ In each subsequent iteration a Re-Assignment of our loop variable takes place in the form of `var=iterable[1]`, `var=iterable[2]`, ... and so until we reach the end of our iterable

- Third the keyword `in`, to tell python with which iterable we want to work
- Fourth the iterable which tells Python the values our loop variable should assume during the repetitions of our source code
 - ▶ Iterables can be lists, dictionaries or simple ranges
 - ▶ When working with ranges we use a standard for loop
 - ▶ When working with datastructures like lists or dictionaries we use a for-each loop

- As we saw python for loops need something called iterables
- We already learned that datastructures are called iterables, since they collect multiple elements in one (simple) structure
- Aside of lists, sets, tuples and dictionaries there are some over forms of iterables, but here I want to focus on the most important one `range()`
- We can use the range function instead of a list or dictionary in the creation of our for loops
- This is especially useful when we just need to repeat source code a certain amount of times without having a datastructure available to us we can loop over

- When creating a for loop with the help of the range function we need to create a loop construct as follows:

```
for i in range(16):
```

```
    instructions
```

```
    instructions
```

- Here i is our loop variable or running index that runs from 0 up to 15
- i increases with every iteration by 1
- We don't need to define or declare i anywhere else in our program since python defines it as soon as it encounters the construction of our for loop

- When using range() as iterable in our for loop construction, we can of course make use of the versatility of this function
- We can create simple loops running from 0 up until 15 with:

```
for i in range(16):
```

- We can create loops running from 5 up until 20 with:

```
for i in range(5, 21):
```

- We can create loops running over even numbers from 0 up until 20 with:

```
for i in range(0, 21, 2):
```

- When using loops like this we name our loop variable `i`, which is an abbreviation for index, to indicate that it takes on simple integer values
- Please stick to this notion as well, BUT only for loops that use `range()` as iterable for everything else use more descriptive names for your loop variable
- Otherwise you will only confuse your self later on when creating your scripts

- On the previous slides you saw two different loops and in both cases the instructions were indented after our loop constructor
- We saw something similar already when talking about if statements
- Here the same principle takes place, that means all lines that have the same indentation level after our loop constructor are seen as inside of our loop
- Of course we can have as many lines of code as we want inside our loop and each of these lines would then be repeated in each iteration
- As soon as python encounters a line that starts at the same level as our for, it knows that loop ends at this point and therefore this line is NOT repeated in each iteration

```
for i in range(4):  
    print(i)
```

➔ i is set to 0 and assigned a value of 0

➔ print(0)

```
print("Finished")
```

- When working with for loops we often run some form of mathematical calculations inside of them
- The instructions can be used to calculate sums and similar things
- Therefore it's essential to understand what happens inside a loop with a variable which changes it's assigned value in each iteration of our loop
- Imagine we need to calculate a sum over a specific range of numbers
- In this case not only do we need to formulate a simple calculation that adds each number of our range together we also need a variable that is assigned the result of this calculation

- Let's come back to the last example and create a simple script that fullfills this task once with a loop and once without a loop
- Down below you see two code snippets that fullfill the exact same task by adding the numbers 1 up until 5 to our variable sum

```
sum = 0
```

```
sum = sum + 1
```

```
sum = sum + 2
```

```
sum = sum + 3
```

```
sum = sum + 4
```

```
sum = sum + 5
```

```
sum = 0
```

```
for i in range(1, 6):
```

```
    sum = sum + i
```

- Write a script that contains 3 separate for loops that use the range function as iterable
- The first one should have at least 20 iterations and print out the loop variable as well as the number of the current iteration
- The second one should have at least 30 iterations and sum up all numbers in the set range, e.g. $0+1+2+3+\dots$
- The third one should have at least 15 iterations and display the results of the multiplication of the loop variable with the numbers 2, 3 and 4 ($i * 2$, $i * 3$ and $i * 4$).

- We saw how to use a for loop with the help of a range
- Python also allows you to loop through each entry of other iterables as well
- When working with lists we can simply iterate over them and in these cases our loop variable is assigned the different values of the elements in our list one by one
- The syntax would look like this:

```
for elem in list:  
    Instructions  
    Instructions
```
- When using such a loop construct make sure to use descriptive names for your loop variable

- When using for loops to iterate over lists it doesn't matter what kind of datatypes the elements of the list have
- BUT it makes more sense to avoid any unnecessary if statements if we use such a construct only for lists that contain elements of ONE specific datatype
 - That means for lists that contain only Integers or only Floats or only Strings
- If you're not sure your list fulfills these criteria you would need to include some form of if statements inside your loop to test each element in your list

```
Numbers = [6, 2, 5, 10]
```

```
for num in Numbers:
```

➔ num is set to 6

```
    print(num)
```

➔ print(6)

```
print("Finished")
```

- Create a list that contains at least 20 different numbers that aren't sorted
 - ▶ Don't use range for the generation of the list instead simply define it using [] and typing the numbers in them
- Write a loop that runs over all elements of the list and adds them all up (sum up all elements)
- Write a second loop that tells the user for each element in the list whether it is bigger or lower than a number specified by you
 - ▶ This number should be generated before the start of the loop and you need to use an if clause inside your loop

- We saw how to iterate over the elements of a list and we also know that a dictionary is an iterable as well
- That means we can iterate over dictionaries as well
- However since dictionaries contain key/value pairs instead of simple elements like lists, they behave a bit different
- A loop as shown below would only iterate over the keys of our dictionary and not the corresponding values:

for key in dictionary:

instructions

```
Numbers = {"6":2, "5":10}
```

```
for key in Numbers:
```

➔ key is set to "5"

```
    print(key)
```

➔ print("5")

```
print("Finished")
```

- If we want to work with the values of a dictionary while using a loop we have two ways of doing it
 1. We can loop over the keys and use our loop variable to access the corresponding value inside our loop
 2. We can loop over the values by using the values() method of our dictionary

```
Numbers = {"6":2, "5":10}
```

```
for key in Numbers:
```

➔ key is set to "5"

```
    print(Numbers[key])
```

➔ print(10)

```
print("Finished")
```

```
Numbers = {"6":2, "5":10}
```

```
for val in Numbers.values():
```

➔ val is set to **20**

```
    print(val)
```

➔ print(**20**)

```
print("Finished")
```

- Write a program that creates a dictionary containing at least four key/value pairs
- The values should be numbers
- Write a loop that iterates over the dictionary and compares each value with the number 10
- Tell the user whether the value bigger, smaller or equal to the number 10
- Use print outs that contain the key and the value

- Each of the two versions of for loops have their own uses
- If you want to do something for a specific number of times, without working with a list, you would use the normal for loop with the range() operator
- If you want to work directly with the contents of a list (or other data structure) you would use the „for each“ loop by iterating over the elements of the corresponding list

- We saw how to use for loops to iterate over the elements of one list
- However sometimes we need to iterate over multiple lists at once and in these cases we can use the zip() function to generate an iterator that aggregates, that means combines elements from two or more iterables
- Please be aware that the function zip() in itself simply generates an iterator object, similar to the range objects of our function range()
- However we can use zip in a loop constructor to iterate over multiple lists in parallel (next slide)
- A nice advantage of using zip() is that it stops as soon as it hits the end of the shortest list we used as argument inside the brackets

- We can use for loops in combination with zip to loop over multiple elements as well:

```
foo = [1, 2, 3]
```

```
bar = [4, 5, 6, 7]
```

```
for f, b in zip(foo, bar):
```

```
    print(f, b)
```

- When using zip() you need multiple loop variables (f and b) and of course you need exact the same amount of loop variables as you have arguments inside of your zip() function

- Write a program that creates three different lists of different lengths
- Loop over all possible two-fold combinations with one for loop each by using zip
- Print out both elements of the lists during each iteration

- When you loop over an iterable it's very easy to get the values of the iterable:
 - `for value in iterable:`
- Sometimes you need a running index in your loop instead of the elements:
 - `for idx in range(len(iterable)):`
- Both work just fine, but sometimes you need a running index and you want to work with the elements of your iterable at the same time
- Of course you could use the second loop construct and everytime you need to work with the value of an element you could use:
 - `iterable[idx]`

- As you can imagine it's very easy to make a mistake here and your code gets complicated relatively fast
- A better way to work with running indices and the elements of your iterable at the same time is to use the function `enumerate()` in your loop construction to get both at the same time:
 - ▶ `for idx, value in enumerate(iterable):`

```
list1 = [5,7,19]
```

```
for idx,value in enumerate(list1):
```

idx=0 and value=5

```
    print(f"Index: {idx}, Value: {value}")
```

print(Index: 0, Value: 5)

```
print("Finished")
```

- The `enumerate()` function has the optional argument `start`, that allows us to define a value with which the running index starts:
 - `for idx, value in enumerate(iterable, start=100):`
- In the example above the value for `idx` would start at 100 instead of 0 which is the default value
- Like this you can adjust the values of your running index inside your loop

- Write a program that defines three different lists
 - They should contain different datatypes
- Create three loops, one for each list, with the help of enumerate
- Try different start values for the index value

- Sometimes we need to make sure that some or all elements of a datastructure fulfill certain criteria
- In these cases we could use some functions in combination with if statements to deal with such checks and they are called:
 - any()
 - all()
- These functions work in combination with loop constructs to fulfill their task

- Returns True if any of the elements of a given iterable (List, Dictionary, Tuple, set, etc) is True
- Syntax: `any(iterable)`
- Parameters: An iterable object such as a dictionary, tuple, list, set, etc.
- Returns: True if any of the items is True
- We can use any in if statements to check whether at least one element of a datastructure assumes a certain value
- For example we could test for elements that are bigger than 6:

```
list1 = [5,6,7,1,5,8]

if any(elem > 6 for elem in list1):

    print("At least one bigger number")
```

- Returns True if all the elements of a given iterable (List, Dictionary, Tuple, set, etc) is True
- It also returns True if the iterable object is empty
- Syntax: `all(iterable)`
- Parameters: An iterable object such as a dictionary, tuple, list, set, etc.
- Returns: True if all of the items are True
- We can use any in if statements to check whether all elements of a datastructure assume a certain value
- For example we could test for elements being bigger than 0:

```
list1 = [5, 6, 7, 1, 5, 8]
if all(x>0 for x in list1):
    print("Only positive numbers")
```

Exercise 38: Working with any() and all()

92

- Run the following checks on the given list using any() and all():
 - ▶ Is there at least one element greater than 10?
 - ▶ Is there at least one element smaller than -5?
 - ▶ Is there at least one element that is equal to 20?
 - ▶ Is there at least one element that is NOT equal to -10?
 - ▶ Is every element negative?
 - ▶ Is every element positive?
 - ▶ Is every element greater than 10?
 - ▶ Is every element equal to 15?
 - ▶ Is every element in a range of -30 to 30? (Bonus)
- `list = [10, 20, -3, -4, 8, 1, 5, -25, 5, 6, -7, 9, 2, 15]`

- List comprehensions offer a shorter syntax when you want to create a new list based on the values of an existing list
- Let's say you have a list of fruits and now, you want a new list, containing only the fruits with the letter "a" in their name:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

- Without list comprehension you will have to write a for statement with a conditional test inside:
- With list comprehension you can do all that with only one line of code:

```
newlist = []  
for x in fruits:  
    if "a" in x:  
        newlist.append(x)  
print(newlist)
```

```
newlist = [x for x in fruits if "a" in x]  
print(newlist)
```

- The general syntax for list comprehensions is as follows:
 - ▶ `newlist = [expression for item in iterable if condition == True]`
- The expression refers to how each element of the original iterable should be modified before it's added to newlist:
 - ▶ For example we could use `x.upper()` in our previous example to convert all elements that fulfill our condition to all upper case versions
 - ▶ Of course we can leave the elements intact by simply using the name we gave to item
- The item is an equivalent to a loop variable and a placeholder
 - ▶ It takes on the value of each element in our iterable one by one
 - ▶ That means in our example `x` has first a value of “apple”, than of “banana” and so on

- The iterable can be any iterable object, like a list, tuple, set etc.
 - ▶ In our example this is the list fruits over which we iterate here
- The condition is like a filter that only accepts the items that fulfill the criteria we specify here
 - ▶ The condition `if "a" in x` in our example, returns True for the elements “apple”, “banana” and “mango”
 - ▶ It can be omitted, that means it is optional

- Fulfill the following tasks with list comprehensions by using the first list given:
 - Get a list with values that are greater than 10
 - Get a list with values that are between 0 and 10, borders excluded
 - Get a list with all negative values
 - Get a list with values smaller -50 and bigger 50
- Optional (use the string list for this):
 - Get a list with strings that contain the word “Hello”
 - Get a list with strings that have at least 7 letters
- Optional (use numerical list):
 - Get a list with multiples of 5 only

- `[10, 250, -5, 2, 3, 84, -11, 0, 6, 24, 115, 3, -80]`
- `["Hello", "HelloWorld", "Christoph", "Hi", "My Name is", "How are you", "Also Hello", "Nice to see you", "Chris", "Nice"]`

- Sometimes you need to repeat a process multiple times, but you don't know the exact number of iterations you need to reach your goal
 - For example you have a data collection about certain organisms and you only want to iterate over them until you find the first organism that fulfills a certain criterium
- In these cases we can use while loops instead of for loops
- When constructing a while loop we need to use a predicate similar to the ones we used for our if statements
- The instructions inside our loop are then repeated until predicate is no longer true
- We need to define a loop variable / control variable before our loop and we need to update it in each iteration of our loop to avoid creating endless loops

- When constructing while loops we have to go over more steps than for for loops
- First we need to define a loop variable, which is sometimes also called control variable we want to use inside our predicate
- Afterwards we define our predicate and create our while loop similar to how we created if statements:

define loop_variable

while (predicate):

- Now we can add our instructions to the loop, which are later repeated until our predicate becomes False
- The most essential step when creating our while loops aside of specifying our predicate is the update of our loop_variable so that we can reach a point on which our predicate becomes False:

```
define loop_variable
```

```
while (predicate):
```

```
    instructions
```

```
    update loop_variable
```

- Let's go over the construction of a while loop step by step
- First we need to define our loop variable, this can be done with a simple variable declaration:

```
i = 0
```

- Second we need to create our while loop with the help of a predicate
- For example we can repeat the code in our loop for as long as the value of our loop variable *i* is smaller than 3:

```
i = 0
```

```
while i < 3:
```

- Third we need some instructions in our loop
- For our example a simple print statement is more than enough

```
i = 0
```

```
while i < 3:
```

```
    print(i)
```

- Fourth we need to update our loop variable, for example by increasing its value by one in each iteration:

```
i = 0
```

```
while i < 3:
```

```
    print(i)
```

```
    i = i + 1
```

```
i = 0
```

→ i contains the value 0

```
while i < 3:
```

→ Question: $i < 3$?

```
    print("Still in Loop")
```

```
    i = i + 1
```

→ value of i increased by 1 → $i=1$

```
print("Loop is finished")
```

- Write a program that uses while loops to finish the tasks below:
 - ▶ Dividing a number of your choice (> 100) by 2 until it's smaller than 5 (as long its bigger than 5)
 - ▶ Adding up the elements of a list of your choice until the sum is bigger than a threshold of your choice

- When talking about if statements you learned that you could nest one if statement into another
- Of course we can also nest one loop into another, without the need of both loops being of the same type (for and while)
- Nevertheless in most cases we nest multiple for loops into each other when we need to repeat a block of repeating source code
- Let's say you have a list of ten protein sequences and now you need to calculate something for each amino acid in these sequences
- In such a case you would create an outer loop over your list and then an inner loop that iterates over the amino acids of the different sequences

- For simplicity reasons we will discuss the construction of nested loops with the help of two simple loops over a range, but of course you can do similar things with loops over lists or other iterables as well
- In the first step we need to create our outer loop, which determines how often the other loops is repeated later on
- Here for this example we only want two iterations so we create the outer loop like this:

```
for i in range(2):
```

- After we created our outer loop we can now add some instructions to it, for example a simple print command
- Don't forget to always add at least one line source code to your loop to avoid any errors:

```
for i in range(2):  
    print(i)
```

- Now it's time to set up the inner loop of our construct
- Be aware that we can NOT use the same loop variable as for our outer loop to make sure everything works as intended

- Per international convention when using the loop variable `i` for an outer loop, we use the next letter in the alphabet as name for the loop variable of the inner loop:

```
for i in range(2):
```

```
    print(i)
```

```
        for j in range(3):
```

- Of course if we nest even more loops into each other we simply go on with the alphabet by using `k`, `l`, `m`, `n` and so on
- Just like we saw for `if` statements we need to make sure that our inner loop has the same indentation level as the remaining instructions we want to be repeated during the iteration of the outer loop

- In the last step we now add the instructions to our inner loop which are repeated for each iteration:

```
for i in range(2):
```

```
    print(i)
```

```
    for j in range(3):
```

```
        print(i,j)
```

- In the construct above the command `print(i,j)` is repeated a total of 6 times, 3-times for the first iteration of the outer loop and 3-times for the second iteration of the outer loop, while the command `print(i)` is repeated only 2-times, one for each iteration of the outer loop

- If we look again at the loop we constructed earlier we can see that the command `print(i)` depends only on the outer loop, while the command `print(i,j)` depends on both loops
- When Python encounters nested loops, it goes to the deepest level (the most inner loop) and repeats this loop as often as needed
- Only afterwards we go to the next iteration of the outer loop

```
for i in range(2):  
    print(i)  
    for j in range(3):  
        print(i,j)
```

```
for i in range(2):
```

➔ i is set to 0

```
    print(i)
```

➔ print(0)

```
        for j in range(3):
```

➔ j is set to 0

```
            print(i,j)
```

➔ print(0,0)

```
print("Loops are Finished")
```

- Write a Program that uses nested loops to tackle the tasks below:
 - ▶ Loop through a two-dimensional list and print out each element
 - ▶ Loop through a two-dimensional list and add up all elements
 - ▶ Loop over two ranges and multiply the loop variables with each other

Exercise Taxes

Visualizing Source Code

- When you want to create more complex programs it's always a good idea to sit down and think about the general construction of your program and the needed logic in your algorithm
- In these cases you should write down the essential parts of your algorithm, that means if statements, loops and so on in a form you can later use when typing down your algorithm
- A very useful tool for this kind of task is Pseudocode, which can help you to visualize the general outline of your program without the need of writing down complete programming constructs

- Displays exemplary certain parts of an algorithm without going into much detail about the needed commands
- Contains only parts of the programming language that are essential for understanding the logic and the general order of events in your algorithm
- It's a simplified version of programming languages that can't be translated into machine code
- There is no standard for Pseudocode, that means the most important thing is that you understand your own pseudocode

- I personally used Pseudocode during my phd-thesis when developing new algorithms for specific tasks
- Since I needed to commute one hour every day I used the time in the afternoon to sit down in the train and write down the general outline of my new function / algorithm I wanted to implement the next day
- At the next day I would use my notes when typing in the program and I didn't need to think about the logic or order of commands any more and instead could focus on typing everything correctly and quickly

- Another way of using Pseudocode I found especially useful was when discussing things with my supervisor
- Since my supervisor had no programming experience at all, but was very knowledgeable about protein structures in general, I could use Pseudocode to discuss my thought processes with her to make sure the biochemical logic was correct
- So Pseudocode can help you when you need to discuss how to tackle a certain task with people who have no or only small programming experience

- It can help you sort your thought process before you start programming
- It allows you to implement your solution later on faster and often times clearer
- It makes discussing algorithms with others easier, since you only focus on the general logic and not the correct implementation
- It allows you to detect eventual logic errors easier after the implementation, when your program produces unwanted results
 - ▶ Even if you don't find an error there, it can help to find positions in your algorithm where you can check the values of your intermediate results for example

- Since there is no standard for Pseudocode it's up to you how you write it
- When trying to illustrate if statements I personally used two different ways:
 1. Check for length:
 - OR
 2. if `len(list) > 10`:
- Both ways are good examples for Pseudocode BUT they do NOT need to be the ones you use when writing Pseudocode

- When writing Pseudocode for loops I mostly used things like this:

for chain in Chains:

 for amino in residues:

 ...

- So I simplified the construction of my loops without going into detail about type of datastructure I used as iterable
- Furthermore I often used names which describe the biological / biochemical phenomenon instead of my real variable names later on

- Another way to illustrate loops, especially if you need the Pseudocode to talk to someone without programming knowledge could be as follows:

Loop over Chains:

 Loop over AminoAcids:
 do something

- Here just the general logic you want to use is described without the usage of any python syntax

Read in pdb-files

for file in pdb-files:

 read in protein structure

 for chain in protein-chains:

 get coordinates for current chain

 for amino in aminoacids:

 get coordinates of current amino acid

 for atom in atoms:

 ...

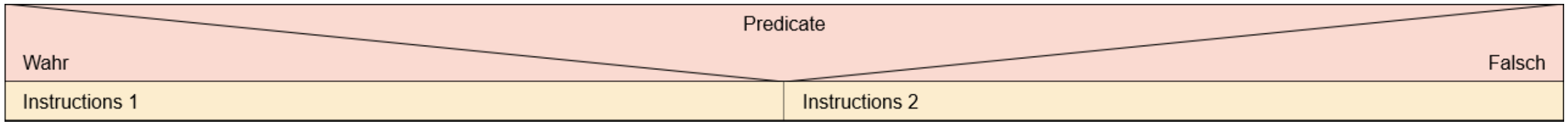
- Write a Pseudocode for the first tasks of:
 - Exercise 34
 - Exercise 40
 - Exercise 41

- Are another option for visualizing Pseudocode especially when you want to publish your algorithm or present it to others
- Different elements of an algorithm (loops, if statements) are represented by specific graphics depending on their type
- Can contain detailed instructions, but this is not necessary
- The scientific name for them is Nassi-Shneiderman-diagrams
- You can use the following web page:
 - <https://ddi.education>

Instruction 1
Instruction 2
Instruction 3
...

The call of functions are simply instructions

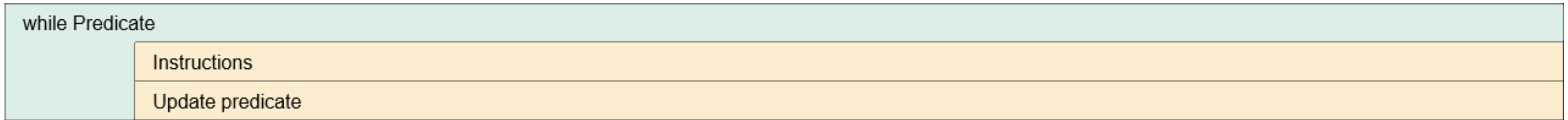
normal if else statement



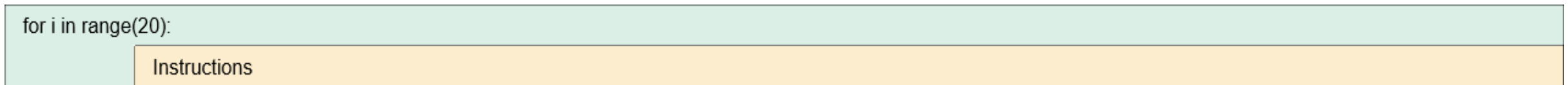
normal if elif else statement

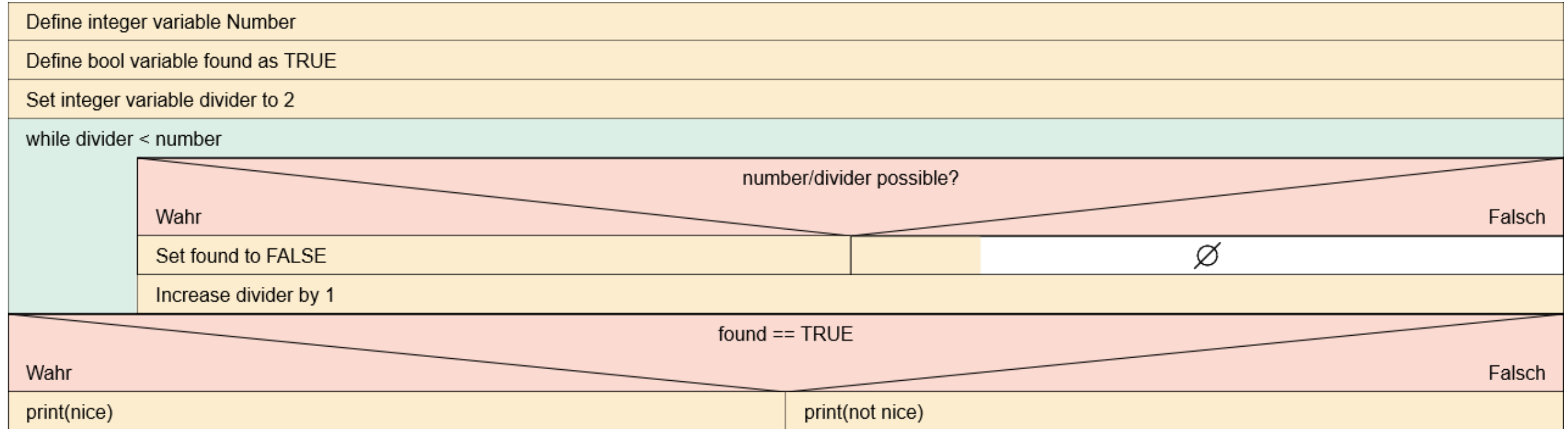


While loop



For loop





- Create some structograms for the first tasks of:
 - ▶ Exercise 34
 - ▶ Exercise 40
 - ▶ Exercise 41

Definitions

- We use **If Statements** to execute only parts of our source code
- **Predicates** are statements that can either be True or False
- **Security Checks** are if statements used to avoid any error messages while running our algorithm
- The process of shifting the start of line of code to the right is called **Indentation** and allows python to recognize which lines are part of an if statement for example
- We can put if statements into another to create **Nested If Statements**
- **Iteration** refers to repeating source code over and over again with the help of a loop construct

- Datastructures are called **Iterables** since we can create loops that take one element at a time and execute the same source code for them
- All loops need a **Loop Variable**, sometimes also called **Control Variable** or **Running Index**, that determines how often the source code inside the loop is repeated
- When using for loops with range, we generally work with the **Running Index i**
- When nesting loops we have an **Outer Loop** that contains other loops as well as **Inner Loops** that are repeated for each iteration of the loop they are part of

1. <https://en.wikipedia.org/wiki/Algorithm> (Last edited: 01.04.2023, Last visited: 04.04.2023)
2. https://en.wikipedia.org/wiki/Muhammad_ibn_Musa_al-Khwarizmi (Last edited: 01.04.2023, Last visited: 04.04.2023)
3. https://en.wikipedia.org/wiki/Ada_Lovelace (Last edited: 29.03.2023, Last visited: 03.04.2023)
4. <https://realpython.com/python-for-loop/> (Last edited: 30.01.2019, Last visited: 06.04.2023)
5. <https://realpython.com/python-while-loop/> (Last edited: 07.11.2018, Last visited: 06.04.2023)
6. <https://en.wikipedia.org/wiki/Pseudocode> (Last edited: 04.01.2023, Last visited: 05.01.2023)
7. https://en.wikipedia.org/wiki/Nassi%E2%80%93Shneiderman_diagram (Last edited: 04.11.2021, Last visited: 03.01.2023)
8. <https://www.geeksforgeeks.org/python-any-function/> (Last edited: 13.9.2021, Last visited: 23.5.2022)
9. <https://www.geeksforgeeks.org/python-all-function/> (Last edited: 15.08.2021, Last visited: 23.05.2022)
10. <https://realpython.com/python-zip-function/> (Last updated: 02.10.2019, Last visited: 13.04.2023)
11. <https://realpython.com/python-enumerate/> (Last updated: 18.11.2020, Last visited: 09.01.2023)

12. https://www.w3schools.com/python/python_lists_comprehension.asp (Last visited: 31.05.2022)